

For the owner

Money, clients and running the studio

You hold every permission, so you can run anything on the other two sheets as well. This one is the part nobody else can do.

If you only remember three

/desk owner

Everything waiting on a decision from you.

/go

The same thing, shorter, with the command for each.

/studio doctor

What is quietly misconfigured, worst first. Run it after any setup change.

Finding your way around

/desk — Your work in one place

/desk owner

Everything waiting on the studio owner

Work and deadlines

/manage — Reassign, hold, cancel and compensate

/manage cancel

Cancel a task, preserving its history

needs task, reason

/manage compensate

Record a payment for work done on a cancelled or reassigned task

needs task, member, amount

/manage flags

Tasks waiting on a decision from you

/plan — Templates, dependencies, blockers and deadline changes

/plan template

Save a reusable task template

needs key, department, label

/plan batch

Create several tasks from a template

needs project, template, count

/review — Internal review and recording client decisions

/review client

Record the client's decision after they replied to you

needs task, decision

/task — Tasks, assignment and pay

/task create

Create a single task

needs project, department, title

/task approve-pay

Approve a leader's proposed pay (owner)

needs task

Money

/bonus — Milestone bonuses: rules, what has been earned, and approvals

/bonus rule-set

Create or change a bonus rule

needs key, label, threshold, amount

/bonus rules

Every bonus rule in the studio

/bonus rule-off

Stop a rule earning anything further (past awards are kept)

needs key

/bonus rule-on

Put a rule back into use

needs key

/bonus pending

Milestones reached and waiting on you

/bonus approve

Agree a bonus is owed

needs id

/bonus decline

Decide a bonus is not owed, with a reason

needs id, reason

/bonus pay

Record an approved bonus as paid

needs id

/deposit — Split what a client owes into named parts, and see what has arrived

/deposit add

Add a part of what the client owes, e.g. "Deposit" or "On delivery"

needs project, label, amount

/deposit list

The parts on an order, and which are covered

needs project

/deposit invoiced

Record that you have asked the client for this part

needs id

/deposit waive

Decide not to charge a part — it stops counting as owed

needs id, reason

/deposit remove

Delete a part that was entered by mistake

needs id

/finance — Client receipts, staff payouts and the studio ledger

/finance client-receipt

Record money received from a client

needs project, amount

/finance pay

Record a payout to someone who worked on the task

needs task

/finance pay-split <i>needs task, share</i>	Record a payout of a finder, leader, mod or owner share
/finance splits <i>needs task</i>	Show how a task's leftover pool divides
/finance set-pool <i>needs task, amount, currency</i>	Enter the distributable pool by hand when currencies differ
/finance mark-payable <i>needs task, reason</i>	Make approved work payable before the client payment lands
/finance ledger	Money in and money out, per currency
/finance outstanding	Approved work that still owes somebody money
/finance balance <i>needs member</i>	What a staff member is owed and has been paid
/finance budget <i>needs project</i>	What a project has committed in pay against what the client pays
/finance budget-override <i>needs project, reason</i>	Deliberately allow pay above the recorded client payment on a project
/finance budget-restore <i>needs project</i>	Put the budget guard back on a project
/finance confirm-received <i>needs payment</i>	The person says the money actually reached them
/finance mark-failed <i>needs payment, reason</i>	A payment was sent and did not arrive — they are owed it again
/finance sent	Payments sent that nobody has confirmed arrived
/finance approve-all	Approve every pay figure your leaders have proposed

People and time

/people — Stand-in leaders and offboarding

/people stand-in grant <i>needs person, department, until, responsibilities</i>	Let somebody run a department for a stated period
/people stand-in revoke <i>needs id</i>	End a stand-in arrangement early
/people stand-in list	Stand-in arrangements
/people offboard preview <i>needs person</i>	What they would leave behind — changes nothing
/people offboard start <i>needs person</i>	Record the departure and take them off the boards
/people offboard complete <i>needs id</i>	Mark an offboarding finished once nothing is outstanding
/people offboard list	Departures, open ones first

/procedure — Studio procedures and acknowledgements

/procedure set Create a procedure, or replace its text

needs key, title, body

/procedure retire Stop showing a procedure (its acknowledgements are kept)

needs key

/procedure who Who has acknowledged the current version, and who has not

needs key

/profile — Your studio profile: timezone, availability, specialties and hours

/profile assign Put a staff member in a department (managers only)

needs member, department

/recommend — Put somebody forward for a trial, a promotion or a leader role

/recommend list Recommendations waiting on a decision

/recommend decide Accept or decline a recommendation

needs id, decision

/trial — Trial briefs for people the studio is considering

/trial offer Write a trial brief and send it

needs person, title, brief, terms

/trial decide Pass or fail a submitted trial, with feedback

needs code, outcome, feedback

/trial withdraw Withdraw a trial that has not been decided

needs code, reason

/trial list Trials, open by default

Clients and orders

/clients — Client records, access and their dashboard

/clients create Create a client record

needs name

/clients add-account Authorize a Discord account to act for a client

needs client, account

/clients revoke-account Remove an account's access

needs client, account

/clients link Attach a project to a client and their channel

needs project, client

/clients dashboard Post or refresh a project's client dashboard

needs project

/clients view A client record, their accounts and order history

needs client

/clients list	All client records
/clients preview <i>needs project</i>	See exactly what a client can see on a project
/clients requests	Open client requests and questions
/clients resolve <i>needs id, status</i>	Close a client request with a note
/clients duplicates	Possible duplicate client records
/clients requirements <i>needs client</i>	Standing requirements this client has on every order
/clients add-requirement <i>needs client, label, detail</i>	Record something this client always wants
/clients retire-requirement <i>needs id</i>	Stop showing a standing requirement (it is kept on the record)
/deliver — Release finished work to a client	
/deliver release <i>needs task</i>	Authorize delivery of an item to the client
/deliver pending	Approved work that has not been released yet
/deliver conditions	What must be true before work can be released
/enquiry — Incoming work enquiries and quotes	
/enquiry new <i>needs service</i>	Record an enquiry that came in outside the bot
/enquiry list	The enquiry pipeline
/enquiry view <i>needs enquiry</i>	One enquiry and its quotes
/enquiry needs-info <i>needs enquiry, question</i>	Mark that you are waiting on the client for details
/enquiry draft-quote <i>needs enquiry</i>	Prepare a draft quote from your templates (still needs owner approval)
/enquiry approve-quote <i>needs enquiry</i>	Approve a draft quote so it can be sent (owner only)
/enquiry send-quote <i>needs enquiry</i>	Record that the approved quote has been sent to the client
/enquiry accept <i>needs enquiry</i>	Client accepted: turn the enquiry into a project with its tasks
/enquiry decline <i>needs enquiry, reason</i>	Record that the enquiry did not go ahead
/enquiry template <i>needs department, unit_price</i>	Set a pricing template for a service

/enquiry templates	Current pricing templates
/outreach — Automated client messages: templates, queue and history	
/outreach template-set <i>needs key, label, body, kind</i>	Write or rewrite a message template (it starts as a draft)
/outreach template-approve <i>needs key</i>	Approve a template so it can actually send
/outreach templates	Every template and whether it is approved
/outreach preview <i>needs key</i>	See a template with sample values, without sending anything
/outreach queue	Messages waiting to go out, and ones that were held back
/outreach cancel <i>needs id</i>	Cancel a queued message before it sends
/outreach history <i>needs client</i>	What has been sent to a client
/outreach prefs <i>needs client</i>	Set a client's messaging rules
/outreach replies	Clients who wrote in and are waiting on an answer
/outreach handled <i>needs id</i>	Mark a client reply dealt with, which lets automation resume
/outreach offer <i>needs project</i>	Queue the approved cross-service offer for one order
/project — Client projects and their tasks	
/project create <i>needs name</i>	Create a project
/project bulk <i>needs project, spec</i>	Turn a bulk order into tasks, e.g. "12 models, 4 vfx, 2 animations"
/project edit <i>needs project</i>	Change a project field
/repeat — Order a client something like last time	
/repeat from <i>needs project</i>	Start a draft from a previous order
/repeat show <i>needs id</i>	See a draft and what is still to confirm
/repeat list	Drafts waiting to be confirmed
/repeat scope <i>needs id</i>	Confirm what is in the order, changing it if needed

/repeat price <i>needs id, amount</i>	Confirm what the client is paying this time
/repeat deadline <i>needs id, date</i>	Confirm the deadline for this order
/repeat create <i>needs id</i>	Turn a fully confirmed draft into a real order
/repeat discard <i>needs id</i>	Throw a draft away

Files and proof

/archive — Search past work and manage portfolio permission

/archive rights <i>needs project, staff, studio</i>	Record what the client allows for portfolio use
/archive asset-rights <i>needs asset, staff, studio</i>	Override portfolio permission for one file
/archive roblox-id <i>needs asset, id</i>	Record the Roblox asset ID a file was uploaded as

/proof — Keep screenshots as proof, and produce the full record of an order

/proof add <i>needs file, kind</i>	File a screenshot or document as proof
/proof list	What has been filed
/proof show <i>needs id</i>	Get a filed file back
/proof verify	Check every filed file is still the one that was filed
/proof record <i>needs project</i>	The whole record of an order, as a file — for a dispute or a chargeback

Keeping an eye on things

/escalate — Raise a concern privately, or handle raised concerns

/escalate list	Concerns raised by staff (owner only)
/escalate respond <i>needs id, action</i>	Acknowledge or resolve a raised concern (owner only)

/report — Filters and reports over the studio's own records

/report filter <i>needs which</i>	Work matching one plain question
/report overview	How many tasks match each filter right now

/report waiting <i>needs project</i>	Where an order's time has gone — with the studio, or with the client
/report person <i>needs member</i>	A rounded picture of one person's work — not a score
/report team	Everybody's picture, in name order
/report payouts	Everyone owed money on approved work
/report attention	Orders that are overdue or have stalled with us
/summary — Management summary and reminder controls	
/summary now	Show the weekly summary right now, privately
/summary post	Post the weekly summary to its channel now
/summary schedule <i>needs cron</i>	Change when the weekly summary is posted
/summary run-reminders	Run the reminder sweep now instead of waiting for the next one

Setting the studio up

/automation — Rules that watch for a situation and tell somebody	
/automation set <i>needs key, label, when, then</i>	Create or change a rule (it starts switched off)
/automation list	Every rule, and whether it is on
/automation preview <i>needs key</i>	What a rule would do right now — it does none of it
/automation on <i>needs key</i>	Switch a rule on
/automation off <i>needs key</i>	Switch a rule off
/automation delete <i>needs key</i>	Remove a rule entirely
/backup — Backups you can restore from, and exports you can read	
/backup now	Take a consistent copy of the database, right now
/backup export <i>needs what</i>	Download readable data as a spreadsheet file
/backup verify	Check a backup file is really a database that opens
/backup restore	How to put a backup back — read this before you need it
/studio — Configure the studio bot (owner only)	
/studio setup	First-run setup: claim ownership and create the default departments

/studio status	Show the current configuration
/studio doctor	Find what is quietly misconfigured or stuck
/studio channel <i>needs purpose, channel</i>	Set one of the bot channels
/studio roles	Set the owner and manager roles
/studio department <i>needs key</i>	Create or update a department and its role mapping
/studio departments	List departments and their role mappings
/studio splits <i>needs finder, leader, mod, owner</i>	Set how the leftover pool is divided (must total 100%)
/studio reminders	Reminder timings and studio-wide quiet hours
/studio currency <i>needs currency</i>	Set the default currency for new projects
/studio capability <i>needs mode, role, capability</i>	Grant or revoke a capability for a role
/studio capabilities	Show which roles hold which capabilities
/studio refresh	Rebuild the staff boards now
/studio sample <i>needs action</i>	Create or remove a demonstration project showing the whole workflow

/web — The studio website: services, pages, client logins and publishing

/web identity	Your studio's name and tagline, as the site shows them
/web service <i>needs key, name, summary</i>	Write or rewrite one service on the public site
/web publish-service <i>needs key, published</i>	Show or hide a service on the public site
/web page <i>needs key, title, body</i>	Write a page, such as About
/web publish-page <i>needs key, published</i>	Show or hide a page
/web status	What the public site currently shows
/web client-email <i>needs client, email</i>	Let a client sign in to the website with an email address
/web revoke-email <i>needs client, email</i>	Remove an email address's access
/web sign-in-link <i>needs client, email</i>	Make a one-time sign-in link to send a client yourself

Where things show up

/desk owner

Everything waiting on a decision from you: pay to approve, quotes to price, client decisions to record, money to release.

Private to you. Start here.

/go

The short version of the same thing — what is waiting on you right now, with the command for each.

The staff info board channel

Every department's board. Set it with `/studio channel purpose:Staff info board`, set how often it rebuilds with `/studio reminders board_refresh_minutes:`, and force a rebuild with `/studio refresh`.

The boards have no command of their own — they post and refresh themselves once the channel is set.

The weekly summary channel

The management digest, posted on the schedule you set with `/summary schedule`. `/summary now` shows it without posting.

The audit log channel

A running record of what was changed and by whom.

Everything is written to the database regardless; this channel is just the readable copy.

The private staff channel

Where a DM goes when somebody's DMs are closed, so a notification is never lost without you knowing.

Each client's own channel

Their dashboard, posted and refreshed by `/clients dashboard`. It shows only what you have released to them.

Check it with `/clients preview`, which shows you exactly what they can see.

The website

The public pages, and the client area where a client who is not in Discord can sign in. `/web status` says what is currently live.

Inside the bot

Screenshots filed with `/proof add` are kept as files on the server, not as Discord links. Get one back with `/proof show`.

They travel with your backups. `/backup now` copies them next to the database.

Setting the channels up

The bot does not create channels. You make them, then point the bot at each one with `/studio channel`. Everything below except the first is staff-only — put them in a category the client roles cannot see.

CYLOPS · STAFF

`#studio-board` — Staff info board

`#studio-alerts` — DM fallback (private staff channel)

`#studio-summary` — Weekly management summary

`#studio-audit` — Audit log

#studio-board

One auto-refreshing message per department. Everyone on the team can read it; nobody types in it.

All staff can read. Nobody needs send access. · Point the bot at it with `/studio channel purpose:Staff info board`

#studio-alerts

Where a notification lands when somebody's DMs are closed, so nothing is lost silently.

All staff. This is why it must not be client-visible. · Point the bot at it with `/studio channel purpose:DM fallback (private staff channel)`

#studio-summary

The digest, on the schedule you set with `/summary schedule`.

You and your leaders. · Point the bot at it with `/studio channel purpose:Weekly management summary`

#studio-audit

A readable record of what changed and who changed it.

You only. It names pay figures and client decisions. · Point the bot at it with `/studio channel purpose:Audit log`

Client channels

Clients get a channel each, outside this category, and you link it with `/clients dashboard`. A client sees only their own channel and only what you have released. Check with `/clients preview`.

Worth knowing

The bot records money, it never moves it

And it never asks for a payment password, card number or wallet phrase. If anything ever does, it is not this bot.

Nothing is payable before the money is in

Unless you say so with `/finance mark-payable`, with your reason kept on the record.

Back up somewhere else

`/backup now` writes to the same disk as the database. That is a second copy, not a backup — see the hosting notes for the offsite job.

159 commands on this sheet, of 226 the bot has. The rest are on the other sheets — the full reference is `docs/COMMANDS.pdf`. Generated 2026-09-30 from the bot itself; rebuild with `npm run reference`.